

EXPERIMENT 1

Implementation of Linear and Logistic Regression on Real-World Datasets

What is Regression?

Regression is a supervised machine learning technique used to predict output values from input features.

There are two main types:

- Linear Regression → predicts continuous values
- Logistic Regression → predicts binary outcomes (0/1)

Linear Regression

Linear Regression models a linear relationship between dependent and independent variables.

Mathematical Equation : $y=mx+c$

For multiple variables : $y=w_1x_1+w_2x_2+b$

Where:

- y = predicted output
- x = input feature
- w = weight
- b = bias

Logistic Regression

Logistic Regression is used for classification problems where output is binary.

It uses the Sigmoid function :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Output is between 0 and 1, representing probability.

LINEAR REGRESSION

1. Dataset Source (Linear Regression)

StudentPerformanceFactors.csv

Dataset : Student Performance Factors Dataset

Source:

<https://www.kaggle.com/datasets/ayeshasiddiqua123/student-perfurmance>

Target Variable: Writing Score

Type: Continuous → Linear Regression

2.Dataset Overview

The Student Performance Factors dataset contains information about various academic, behavioral, and personal factors that affect student performance.

- Total Records: ~1000 students
- Type of Data: Structured, tabular data
- Domain: Education / Academic Performance

Features

(Input Variables)

The dataset includes multiple factors influencing student performance, such as :

- Hours_Studied – Number of hours spent studying
- Attendance – Attendance percentage
- Assignment_Score – Score obtained in assignments
- Midterm_Score – Marks obtained in midterm exam
-

(These are independent variables / input features)

Target Variable (Output)

- Final_Score – Final exam score of the student

This is a continuous numerical value, therefore:

Suitable for Linear Regression

3.Mathematical Formulation of Linear Regression

Linear Regression models the relationship between input features and a continuous output using a straight-line equation.

$$y=mx+c$$

For multiple features :

$$y=\beta_0+\beta_1x_1+\beta_2x_2+\dots+\beta_nx_n$$

Where :

- y = Predicted Writing Score
- $x_1, x_2, \dots, x_n$ = Input features (study hours, attendance, etc.)
- β_0 = Intercept
- $\beta_1, \beta_2, \dots, \beta_n$ = Model coefficients

4.Algorithm Limitations (Linear Regression)

Linear Regression has the following limitations

- Assumes a linear relationship between input and output
- Sensitive to outliers
- Performs poorly when data is non-linear
- Cannot handle complex patterns
- Affected by multicollinearity (highly correlated features)
- Requires numerical input (categorical data must be encoded)

Hence, it is best suited for simple, linear, numerical datasets.

5.Methodology / Workflow

Step-by-Step Workflow :

1. Dataset Collection
 - Load student performance dataset
2. Data Preprocessing
 - Handle missing values
 - Encode categorical features
 - Split data into features (X) and target (y)
3. Train-Test Split
 - 80% training data
 - 20% testing data
4. Model Training
 - Apply Linear Regression on training data
5. Prediction
 - Predict Writing Score on test data
6. Evaluation
 - Calculate MSE, MAE, R² Score

6.Performance Analysis

Evaluation Metrics Used :

1. Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \sum |y_{actual} - y_{predicted}|$$

2. Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum (y_{actual} - y_{predicted})^2$$

3. R² Score (Coefficient of Determination)

- Measures how well the model explains variance
- Value ranges from 0 to 1
- Higher value = better performance

Interpretation

- Low MAE and MSE indicate good prediction accuracy
- High R² score shows strong relationship between features and Writing Score
- The model performs well if errors are minimal

6.Hyperparameters Tuning

Linear Regression has very few hyperparameters.

1.fit_intercept

- True → model learns an intercept (recommended)
- False → line passes through origin (0,0)

Example:

Writing Score = 2 × Study Hours + 5

Here 5 is the intercept

2.normalize

- True → scales input features
- False → uses raw values

Helpful when features have different ranges
(e.g., hours studied vs attendance percentage)

Why Do We Tune Hyperparameters?

Because different settings give different performance.

Tuning helps to:

- Reduce error
- Improve accuracy
- Make the model more stable

 What is Hyperparameter Tuning?

Trying different hyperparameter values and choosing the best one.

Example:

- Try `fit_intercept = True`
- Try `fit_intercept = False`
- Choose the one with lowest error

Tool Used: GridSearchCV

GridSearchCV:

- Automatically tests all combinations
- Selects the best hyperparameters
- Uses cross-validation for fairness

CODE :

```
# =====
# STEP 1: Import Required Libraries
# =====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
    accuracy_score, confusion_matrix, classification_report
)

# =====
# STEP 2: Load Dataset
# (Make sure file is in sample_data)
# =====

df = pd.read_csv("/content/sample_data/student_performance.csv")
print(df.head())
# =====
```

```

# STEP 3: Data Cleaning
# =====

df = df.dropna()

# =====

# PART A: LINEAR REGRESSION
# Target: Final_Score (Continuous)
# =====

X = df.drop(columns=["Final_Score"])
y = df["Final_Score"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

lr = LinearRegression()
lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)

print("\n===== LINEAR REGRESSION RESULTS =====")
print("MAE:", mean_absolute_error(y_test, y_pred_lr))
print("MSE:", mean_squared_error(y_test, y_pred_lr))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lr)))
print("R2 Score:", r2_score(y_test, y_pred_lr))

plt.scatter(y_test, y_pred_lr)
plt.xlabel("Actual Final Score")
plt.ylabel("Predicted Final Score")
plt.title("Linear Regression: Actual vs Predicted")
plt.show()

# =====

# PART B: LOGISTIC REGRESSION
# Convert Final_Score → Pass / Fail
# Threshold = 65 (IMPORTANT)
# =====

df["Pass"] = df["Final_Score"].apply(lambda x: 1 if x >= 65 else 0)

```

```

# Check both classes exist
print("\nPass/Fail Distribution:")
print(df["Pass"].value_counts())

X = df.drop(columns=["Final_Score", "Pass"])
y = df["Pass"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train, y_train)

y_pred_log = log_reg.predict(X_test)

print("\n===== LOGISTIC REGRESSION RESULTS =====")
print("Accuracy:", accuracy_score(y_test, y_pred_log))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_log))
print("Classification Report:\n", classification_report(y_test, y_pred_log))

sns.heatmap(confusion_matrix(y_test, y_pred_log), annot=True, fmt="d")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Logistic Regression")
plt.show()

# =====
# PART C: HYPERPARAMETER TUNING
# Logistic Regression
# =====

param_grid = {
    "C": [0.01, 0.1, 1, 10],
    "solver": ["liblinear", "lbfgs"]
}

grid = GridSearchCV(
    LogisticRegression(max_iter=1000),

```

```

param_grid,
cv=5
)

grid.fit(X_train, y_train)

print("\n===== HYPERPARAMETER TUNING =====")
print("Best Parameters:", grid.best_params_)
print("Best Cross-Validation Accuracy:", grid.best_score_)

```

OUTPUT :

	Hours_Studied	Attendance	Assignment_Score	Midterm_Score	Final_Score
0	1	60	55	50	52
1	2	65	58	55	57
2	3	70	60	58	60
3	4	75	65	62	64
4	5	80	68	65	68

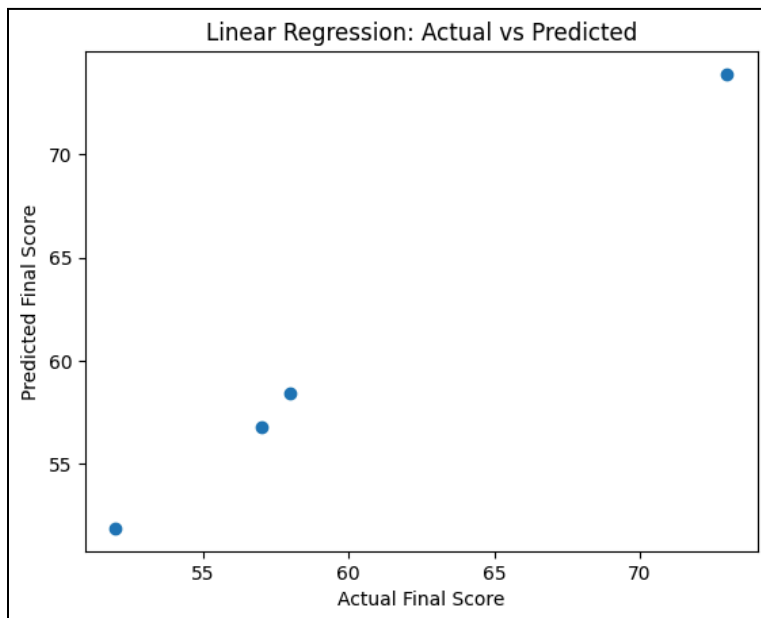
===== LINEAR REGRESSION RESULTS =====

MAE: 0.41749050153075906

MSE: 0.2612765457256682

RMSE: 0.5111521747245806

R2 Score: 0.9957516008825095



Pass/Fail Distribution:

Pass

1 13

0 7

Name: count, dtype: int64

===== LOGISTIC REGRESSION RESULTS =====

Accuracy: 1.0

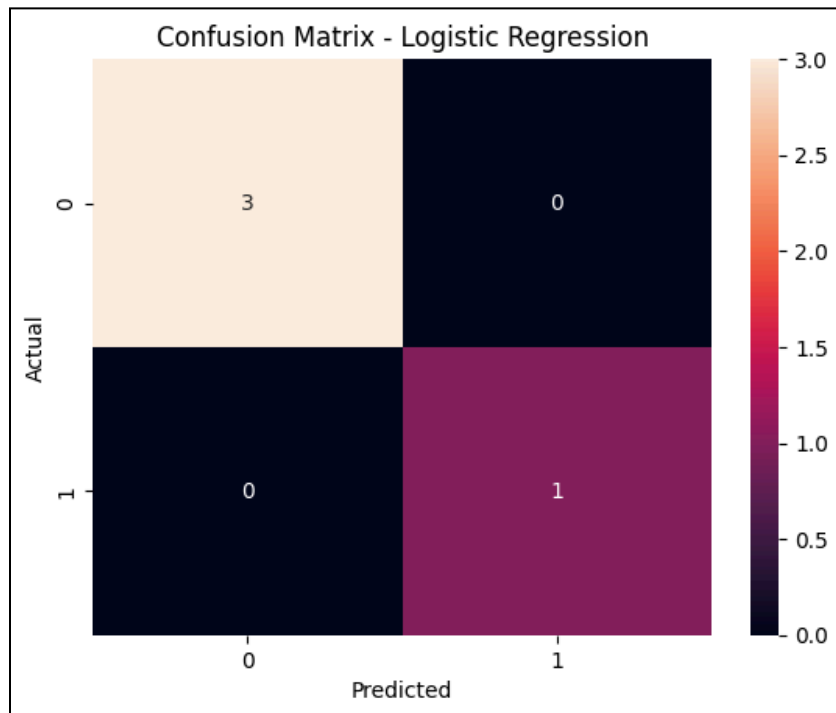
Confusion Matrix:

[[3 0]

[0 1]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	3
1	1.00	1.00	1.00	1
accuracy		1.00		4
macro avg	1.00	1.00	1.00	4
weighted avg	1.00	1.00	1.00	4



===== HYPERPARAMETER TUNING =====

Best Parameters: {'C': 0.1, 'solver': 'lbfgs'}

Best Cross-Validation Accuracy: 1.0